



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Modelação e Simulação de Sistemas Naturais
Trabalho Prático 2

Turma: 31D

Grupo: 30

Nome: Miguel Alcobia

Número: 50746

Nome: Fábio Pestana

Número: 50756

Professor: Paulo Vieira

Data: 26 de Novembro de 2023

Índice

Índice.....	I
Lista de figuras	II
Introdução.....	3
B - Queda livre com atrito (Silico).....	4
Exercício 1	4
Exercício 2	5
Exercício 3	6
Exercício 4	7
C - Sistemas Solar e Sistema de Partículas	8
Exercício 1	8
Exercício 2 e 3	9
D - Agentes Autónomos - Comportamentos individuais.....	12
Exercício 1 - Boid com acelerador e travão.....	12
Exercício 4 - Comportamento Wander.....	12
Exercício 5 - Comutação de comportamento.....	13
E - Agentes Autónomos - Comportamentos de grupo (Flocking).....	14
Exercício 1 - Predador a perseguir uma pressa.....	14
Exercício 3 - Flock com liderança.....	16
Exercício 4 - Debugging.....	17
Conclusão.....	18
Bibliografia.....	19

Lista de figuras

Figura 1 - Diagrama B1	4
Figura 2 - Diagrama B2	5
Figura 3 - <i>Data Table</i> da Posição (B1)	5
Figura 4 - <i>Data Table</i> da Velocidade (B2)	5
Figura 5 - Diagrama B3	6
Figura 6 - <i>Data Table</i> da Posição (B3)	6
Figura 7 - <i>Data Table</i> Velocidade (B3)	7
Figura 8 - <i>Data Table</i> da Força Total (B4)	8
Figura 9 - <i>Print</i> de um planeta em torno do Sol	9
Figura 10 - <i>Print</i> do Sistema Solar (zoom x3)	11
Figura 11 - Comportamento do Sistema Solar com G aumentado 4 vezes	11
Figura 12 - Desenho dos vídeos com anotações dos alunos	13
Figura 13 - <i>Print</i> da BoidApp	14
Figura 14 - Erro de boids a "piscar"	15
Figura 15 - <i>Print</i> da FlockTextApp com um líder	17
Figura 16 - <i>Print</i> da FlockDebuggingApp	17

Introdução

Neste trabalho, os alunos tinham que analisar o comportamento em queda livre; implementar um sistema solar com sistema de partículas atrelado; implementar agentes autónomos (com comportamentos individuais e de grupo). Todas as implementações foram feitas com Processing 3 em Java e o estudo sobre a queda livre foi realizado com a ferramenta *Silico*.

B - Queda livre com atrito (Silico)

Exercício 1

Com o auxílio do *Silico* desenhou-se este diagrama (hiperligação) :

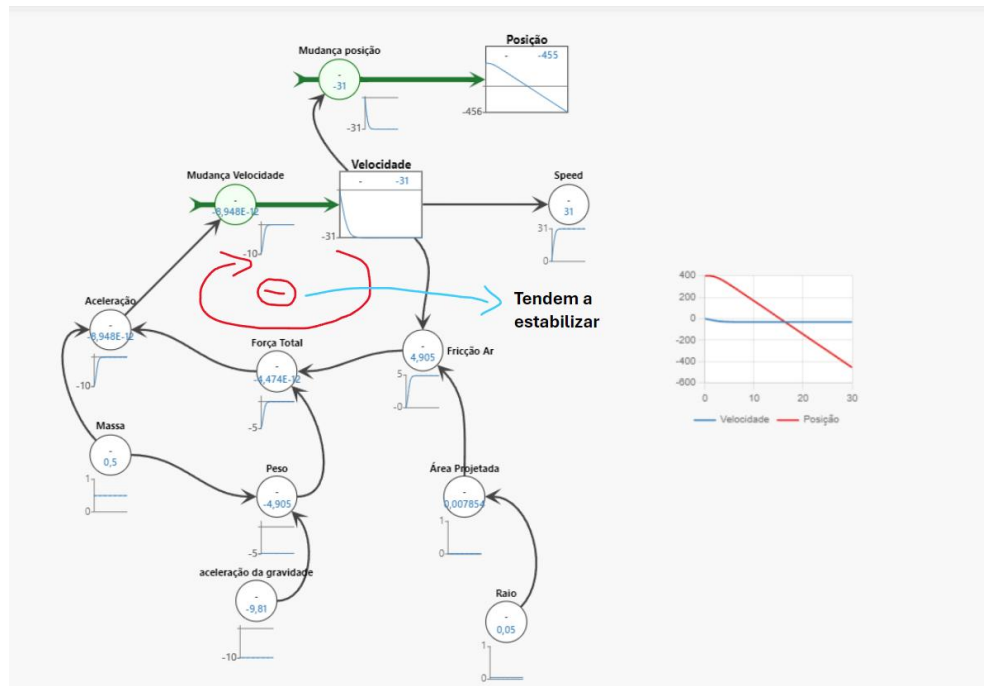


Figura 1 - Diagrama B1

Temos um ciclo de *feedback* negativo entre a “Velocidade”, “Fricção Ar”, “Força Total”, “Aceleração” e “Mudança Velocidade”, que tendem a estabilizar; pois a força de atrito “Fricção Ar” aumentaria durante a queda, tal como a velocidade. Em determinado ponto, a força de atrito e velocidade têm o mesmo valor e fazem com que a “Força Total” seja 0, porque $FT = Peso + FricçãoAr$.

Exercício 2

Novamente com o auxílio do *Silico* desenhou-se este diagrama (hiperligação):

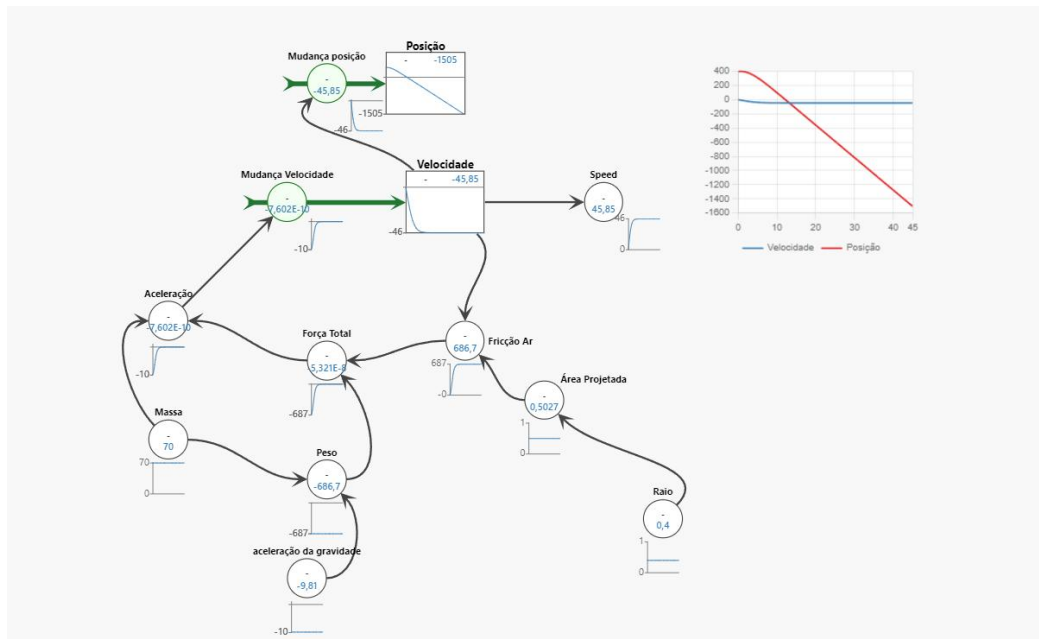


Figura 2 - Diagrama B2

Data		
	Simulated	Actual
5	310,148	-
6	272,004	-
7	230,841	-
8	187,776	-
9	143,558	-
10	98,655	-
11	53,354	-
12	7,821	-
13	-37,845	-
14	-83,587	-
15	-129,373	-
16	-175,105	-

Figura 3 - Data Table da Posição (B1)

O corpo chegou ao solo algures entre os 12 e os 13 segundos. Logo, levou ente 12 a 13 segundos para chegar ao solo.

Data		
	Simulated	Actual
9	-44,902	-
10	-45,302	-
11	-45,533	-
12	-45,666	-
13	-45,742	-
14	-45,786	-
15	-45,811	-
16	-45,826	-
17	-45,834	-
18	-45,839	-
19	-45,841	-
20	-45,843	-

Figura 4 - Data Table da Velocidade (B2)

A velocidade era algo ente 45,666 e 45.742 m/s.

Exercício 3

Novamente com o auxílio do *Silico* desenhou-se este diagrama (hiperligação):

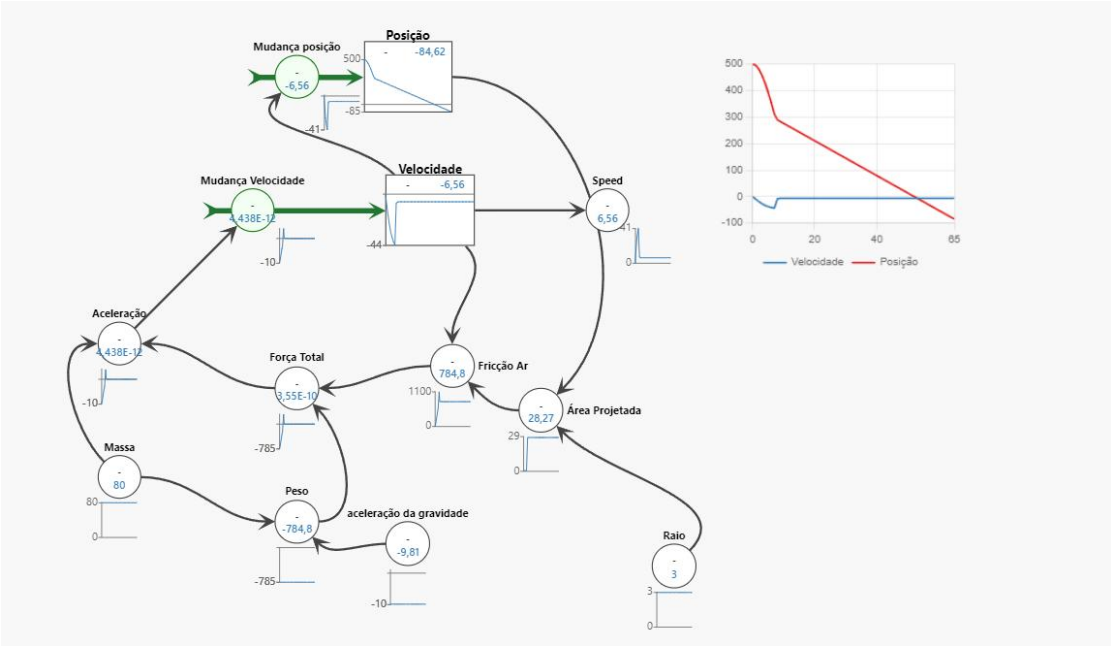


Figura 5 - Diagrama B3

A massa do corpo agora passa a ser 80kg (70 + 10kg) e o raio passa a ser 3m (6/2 m).

O corpo demora entre 52 e 53 segundos a chegar ao solo e atinge-o com uma velocidade 6.56 m/s. O paraquedista cai a pique, com um aumento significativo na velocidade, durante mais ou menos 8 segundos e ao abrir o paraquedas, a velocidade estabiliza e a posição vai variando linearmente.

	Simulated	Actual
47	33,465	-
48	26,905	-
49	20,345	-
50	13,785	-
51	7,225	-
52	0,665	-
53	-5,895	-
54	-12,455	-
55	-19,015	-
56	-25,575	-
57	-32,135	-
58	-38,695	-

Figura 6 - Data Table da Posição (B3)

	Simulated	Actual
48	-6,56	-
49	-6,56	-
50	-6,56	-
51	-6,56	-
52	-6,56	-
53	-6,56	-
54	-6,56	-
55	-6,56	-
56	-6,56	-
57	-6,56	-
58	-6,56	-
59	-6,56	-

Figura 7 - Data Table Velocidade (B3)

Exercício 4

Novamente com o auxílio do *Silico* desenhou-se este diagrama (hiperligação):

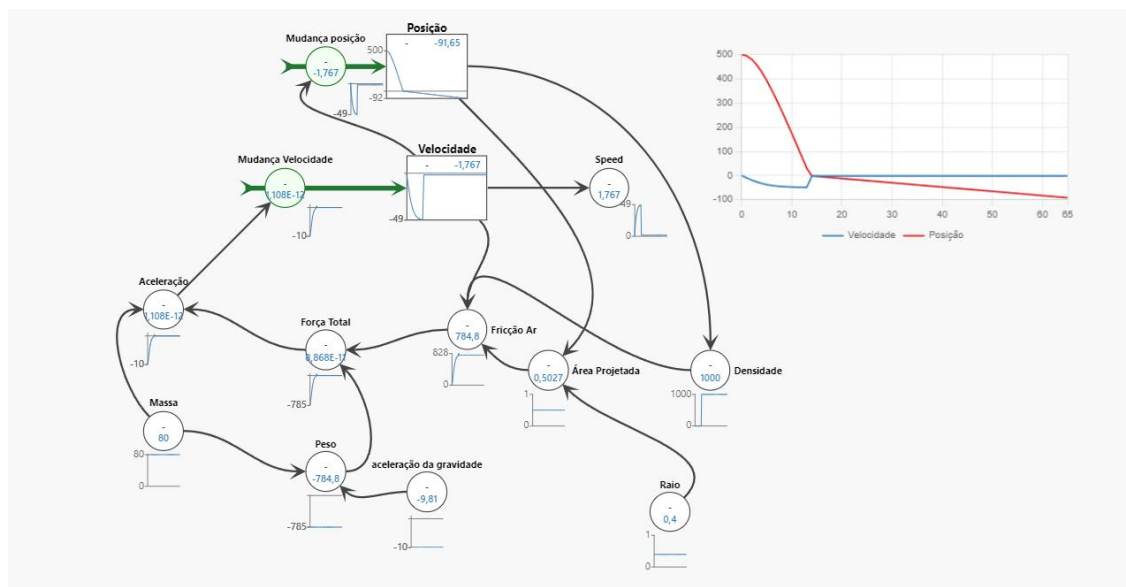


Figura 6 - Diagrama B4

O raio volta a ser 3m (6/2 m) e temos uma condição para a densidade ser 1.29 se a posição estiver acima do solo e 1000 caso contrário.

O corpo demora entre 13 a 14 segundos a chegar ao solo (mais um pouco do que no Exercício 2, devido aos 10kg do paraquedas) e atinge-o com uma velocidade 48.652 m/s. O paraquedista cai a pique e atinge o rio com uma força de 42.484 Newtons.

O paraquedista submerge com uma velocidade de 1.767 m/s.

	Simulated	Actual
12	-25,781	-
13	-17,397	-
14	42,484	-
15	6,195E-4	-
16	9,277E-9	-
17	8,868E-11	-
18	8,868E-11	-
19	8,868E-11	-
20	8,868E-11	-
21	8,868E-11	-
22	8,868E-11	-
23		

Figura 8 - Data Table da Força Total (B4)

C - Sistemas Solar e Sistema de Partículas

Exercício 1

Começou-se por desenvolver a classe `Mover` que será a base para a classe que representará os planetas, o Sol e para a classe que diz respeito ao cinturão de asteroides. Esta classe tem como atributos o vetor de posição (`pos`), um de velocidade (`vel`), um de aceleração (`acc`) e dois valores `float` que dizem respeito à massa (`mass`) e ao raio (`radius`) do corpo, respetivamente.

Esta classe tem dois métodos importantes:

- `applyForce(PVector f)`: Aplica uma força ao corpo. A força é dividida pela massa do corpo antes de ser adicionada à aceleração, segundo a segunda lei de Newton: $F = m\vec{a}$.
- `move(float dt)`: Atualiza a posição do corpo com base na velocidade e na aceleração, consoante um intervalo de tempo `dt`.

Desta forma podemos fazer simulações levando as leis da Física em conta.

A classe `Body` é a classe que representará o Sol e os planetas do sistema Solar e herda os atributos e os métodos da classe `Mover`. Tem como atributos próprios a cor (`int color`), a constante gravitacional $G = 6.67e-11$ (`double G`), e o seu nome (`String name`). Além dos *gets* e *sets* criou-se os seguintes métodos:

- `attraction(Mover m)`: Calcula e retorna o vetor de atração gravitacional entre o corpo e outro objeto `Mover`, que é passado como parâmetro.
- `resetG()`: Restaura a constante gravitacional para o valor padrão.
- `isOver(PApplet p, SubPlot plt)`: Verifica se o rato está sobre o corpo. É usado para se retornar `true`, é exibida a informação do respetivo corpo.
- `displayInfo(PApplet p, SubPlot plt)`: Exibe as informações completas do

corpo (nome, posição, velocidade, massa).

Com estas classes já se criou a App para o Exercício 1, a que se deu o nome PlanetStarApp. Definiu-se os valores necessários para representar o Sol e a Terra:

```
private float sunMass = 1.989e30f;
private float earthMass = 5.97e24f;
private float distancePlanetStar = 1.496e11f;
private float earthSpeed = 3e4f;
```

O setup foi feito da seguinte forma para a simulação:

```
public void setup(PApplet p) {
    plt = new SubPlot(window, viewport, p.width, p.height);
    sun = new Body(new PVector(), new PVector(), sunMass, distancePlanetStar/10,
    p.color(255,128,0));
    earth = new Body(new PVector(0,distancePlanetStar), new PVector(earthSpeed,0),
    earthMass, distancePlanetStar/20, p.color(0,180,120));
```

O resultado foi o que se pode ver no *print* abaixo:

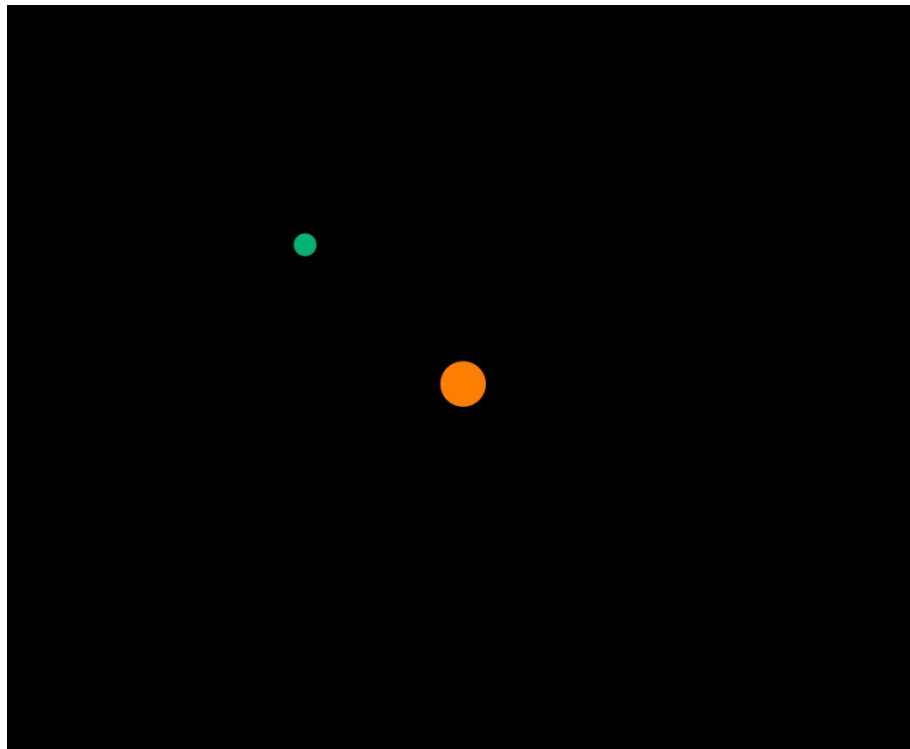


Figura 9 - *Print* de um planeta em torno do Sol

Exercício 2 e 3

Para o Sistema completo foi elaborada a SistemaSolarApp, que inicializa alguns atributos com valores importantes para a simulação:

- `planetasDoSistemaMassa`: Array que armazena as massas dos corpos celestes no sistema solar, incluindo o Sol e os planetas (em kg).
- `planetasRaio`: Array que contém os raios dos corpos celestes do sistema solar, para facilitar a visualização durante a simulação este valores não correspondem á realidade.

- `planetasDoSistemaDist_Sol`: Array com as distância de cada planeta em relação ao Sol (em metros).
- `planetasDoSistemaTranslacao_Sol`: Array com a velocidade de translação de cada planeta ao redor do Sol (em metros/segundo).
- `planetsName`: Array com nomes dos planetas no sistema solar.

Além dos atributos referidos acima, é criado cinco `ParticleSystem`, quatro para representar a cintura de asteroides (asteroides entre Marte e Júpiter), e um para representar as explosões do Sol.

Em relação a métodos:

- `setup()`: Inicializa a aplicação, cria os corpos celestes.
- `draw()`: Desenha os corpos celestes, nas respectivas órbitas, e desenha outros elementos como os botões e as instruções de uso.
- `keyPressed()`: Faz *reset* ao sistema solar (ao pressionar “R”), zoom in no “+” e zoom out no “-”.
- `mousePressed()`: Aumenta ou diminui a gravidade do Sol, consoante o botão pressionado no ecrã.
- `orbitPlanet()`: Método para calcular a atração gravitacional e a órbita do corpo celeste.

O `ParticleSystem` faz *extends* do `Mover` e é formado por várias `Particle`. Cada *particle* segue o trajeto aleatório e vai gradualmente desaparecendo, consoante o seu *lifetime*. Nesta classe temos o método `changeAngleOfSP()` para calcular o ângulo em relação ao Sol que o cinturão de asteroides deve assumir à medida que cumpre a sua órbita.

Eis o *print* do resultado final:

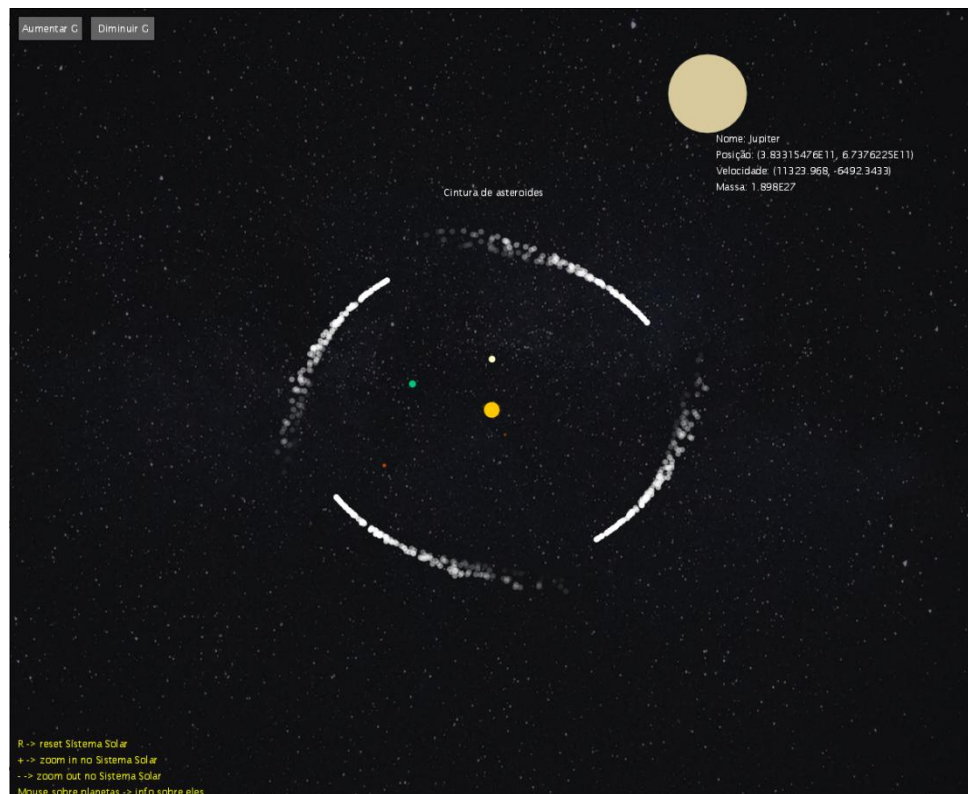


Figura 10 - Print do Sistema Solar (zoom x3)

Ao aumentar G , observamos o comportamento esperado, dos corpos celestes a serem muito mais atraídos pelo Sol. Já se diminuirmos, observamos os corpos a afastarem do Sol, ao ponto de se deixar de ver alguns deles.

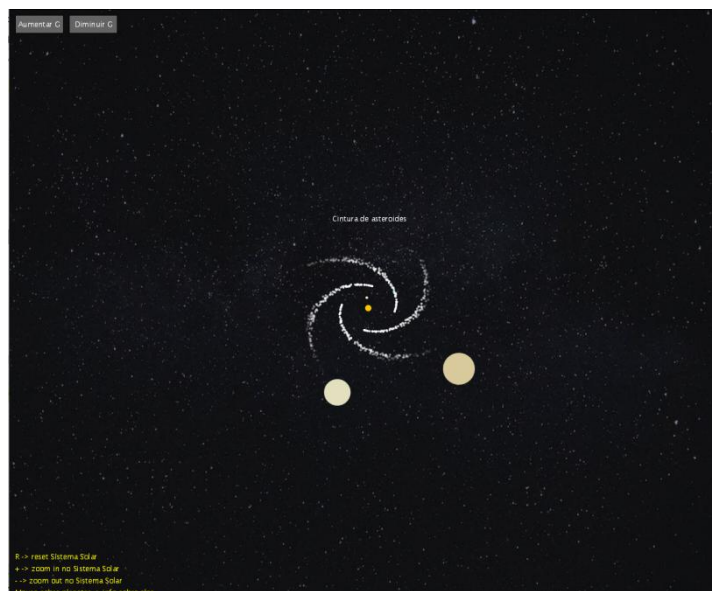


Figura 11 - Comportamento do Sistema Solar com G aumentado 4 vezes

D - Agentes Autônomos - Comportamentos individuais

Exercício 1 - Boid com acelerador e travão

Começou-se por desenvolver a classe `Boid` que deriva da classe `Body`. Esta classe tem como atributos uma `PShape` (`shape`) na forma de uma seta que será o nosso boid, uma lista para os vários Behaviors que o Boid pode assumir, um ADN (`ADNBoid adn`) e um olho (`Eye eye`) para ver à sua volta.

Para o acelerador e travão fez-se o seguinte no `draw()` do `BoidApp`:

```
@Override
public void keyPressed(PApplet p) {
    if (p.key == '+') {
        if (boid.getPos().dist(target.getPos()) < 5) {
            //Map para relacionar o "raio de reação" com a velocidade
            // e quando chegar ao target parar
            float mappedSpeed = p.map(boid.getVel().mag(), maxSpeed[0], 0, 5f, 0);
            float smoothFactor = 0.1f;
            float smoothedSpeed = boid.getVel().mag() + (mappedSpeed - boid.getVel().mag())
            * smoothFactor;
            boid.setVel(boid.getVel().normalize().mult(smoothedSpeed));
        } else {
            float newSpeed = boid.getVel().mag() * 2;
            float limiteVel = Math.min(newSpeed, maxSpeed[0]);
            boid.setVel(boid.getVel().normalize().mult(limiteVel));
        }
    }

    if (p.key == '-') {
        // Desacelera a velocidade do Boid pela metade, mas não vai abaixo de zero
        float newSpeed = boid.getVel().mag() / 2;
        float limiteVel = Math.max(newSpeed, 0); // Garante que não vai abaixo de zero
        boid.setVel(boid.getVel().normalize().mult(limiteVel));
    }
}
```

O acelerador é acionado pelo “+” e por feito com um `map()`, relacionando a posição do target com a velocidade atual do boid, querendo que quando ele chegue ao local do target tenha 0 de velocidade. Este comportamento é ativado quando o boid está a uma distância inferior de 5 do target. O `smoothFactor` foi usado para melhorar a travagem, evitando que ela fosse demasiado brusca.

O botão “-” aciona o travão que diminui a velocidade, mas impede que esta seja inferior a 0.

A implementação deste comportamentos só não foi perfeito, devido ao facto de que o target às vezes não alcança o alvo quando está acelerado, devido às forças aplicadas, embora vá na sua direção. Já o travão funciona perfeitamente.

Exercício 4 - Comportamento Wander

Seguindo as indicações dadas na aula teórica e nos vídeos disponibilizados. O Wander foi implementado como sendo uma classe que herda as características de `Behavior` e tem como método

próprio o `getDesiredVelocity()` e segue a fórmula apresentada na figura seguinte:

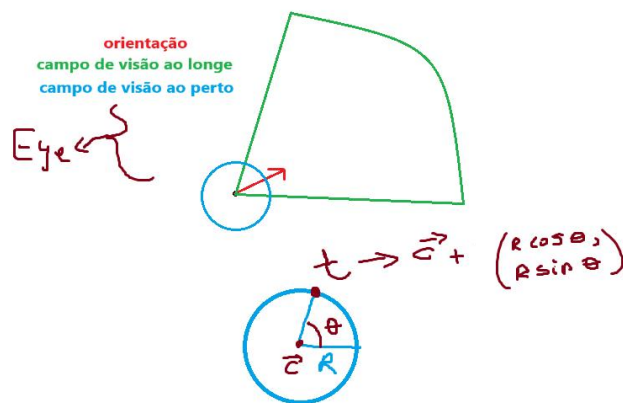


Figura 12 - Desenho dos vídeos com anotações dos alunos

A fórmula foi implementada da seguinte forma:

```
@Override
public PVector getDesiredVelocity(Boid me) {
    PVector center = me.getPos().copy();
    center.add(me.getVel().copy().mult(me.adn.deltaTWander));
    PVector target = new PVector((float) (me.adn.radiusWander * Math.cos(me.phiWander)),
    (float) (me.adn.radiusWander * Math.sin(me.phiWander)));
    target.add(center);
    me.phiWander += 2 * (Math.random() - 0.5) * me.adn.deltaPhiWander;
    return PVector.sub(target, me.getPos());
}
```

Exercício 5 - Comutação de comportamento

A troca de comportamentos foi implementada através de botões, sendo que o target do boid muda de posição sempre que o rato é premido, e dependendo do botão pressionado ele pode fazer *seek* ou *flee* ao target (por *default* ele começa a fazer *seek*), o comportamento *wander* não é afetado pelo target:

```
@Override
public void mousePressed(PApplet p) {
    double[] ww = plt.getWorldCoord(p.mouseX, p.mouseY);
    target.setPos(new PVector((float) ww[0], (float) ww[1]));
    if (isMouseOverButton(p, margin, buttonWidth, margin, margin+buttonHeight)) {
        idx = 0; //Seek
    }
    if (isMouseOverButton(p, buttonx, buttonx+buttonWidth, margin, margin+buttonHeight)) {
        idx = 1; //Flee
    }
    if (isMouseOverButton(p, buttonx2, buttonx2+buttonWidth, margin, margin+buttonHeight))
    {
        idx = 2; //Wander
    }
}
```

O resultado final de todas alienas do exercício D apresentou o seguinte aspeto:

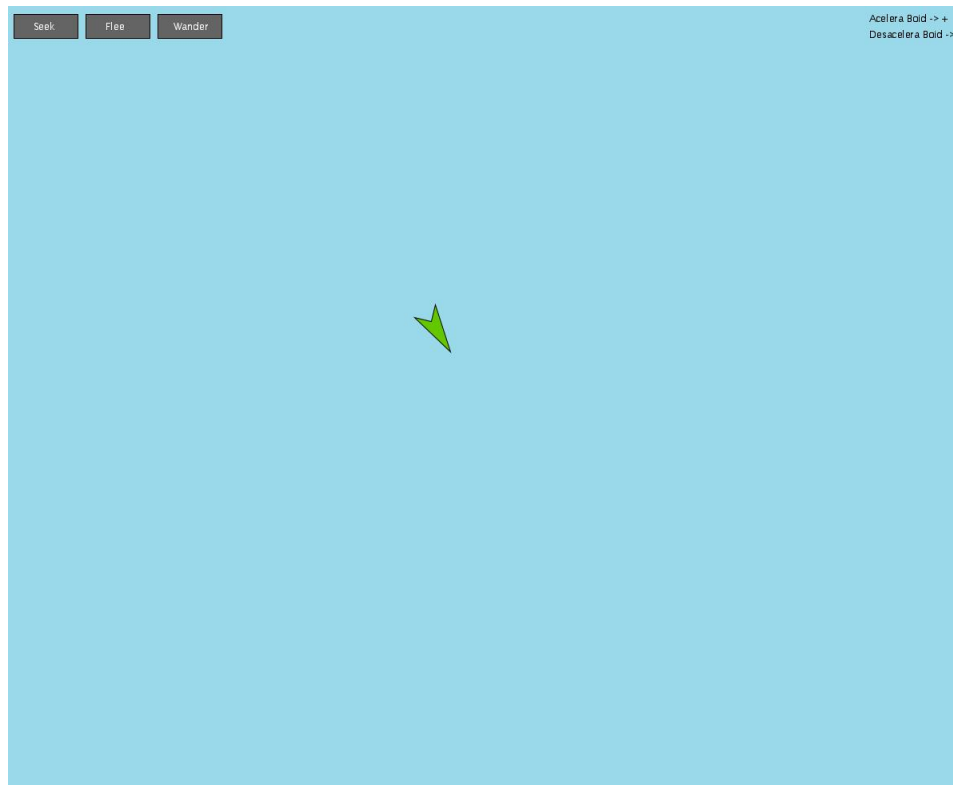


Figura 13 - *Print* da BoidApp

E - Agentes Autónomos - Comportamentos de grupo (Flocking)

Exercício 1 - Predador a perseguir uma presa

Neste exercício o funcionamento da App deve-se sobretudo aos métodos `mousePressed`, `KeyPressed` da `FlockTextApp()`.

O predador e a presa são acionados por um botão no ecrã. Primeiro é preciso seleccionar qual o boid que será o presa com um simples clique sobre o mesmo. Depois o predador tem uma perseguição automática sobre a presa).

O tratamento que se deu ao dados foi o seguinte:

```

public void mousePressed(PApplet p) {
    Boid aux = flock.getBoidMouse(p);
    int i = flock.getBoidIndex(aux);

    (...)

    //Predador/Presa
    if (BoidApp.isMouseOverButton(p, buttonx, buttonx+buttonWidth, margin,
margin+buttonHeight)) {
        flock.getBoid(controlledBoidIndex).setShape(p, plt, p.color(200, 200, 0));
        float[] weights = {1f, 0.5f, 0.5f, 0f, 2f};
        flock.removeAllBehavior();
        flock.setBehaviorAllBoids(weights);
        predador = new Boid(new PVector(), new PVector(), 1, 1f, p.color(100, 200, 0), p,
plt);
        predador.addBehavior(new Seek(2f));
        b=aux;
        allTrackingBodies.add(b);
        eye = new Eye(predador, allTrackingBodies);
        predador.setEye(eye);
    }
    if (i!=-1 && allTrackingBodies.size() !=0) {
        b = aux;
        allTrackingBodies.set(0, b);
        if (predador!=null)
        {
            flock.getBoid(i).setShape(p, plt, p.color(p.random(255), p.random(255),
p.random(255)));
            eye = new Eye(predador, allTrackingBodies);
            predador.setEye(eye);
        }
    }
}
}

```

É de observar que, apesar do comportamento de perseguição funcionar corretamente, os boids do flock das presas ficam a “piscar” e um pouco presos ao tocarem nas margens.

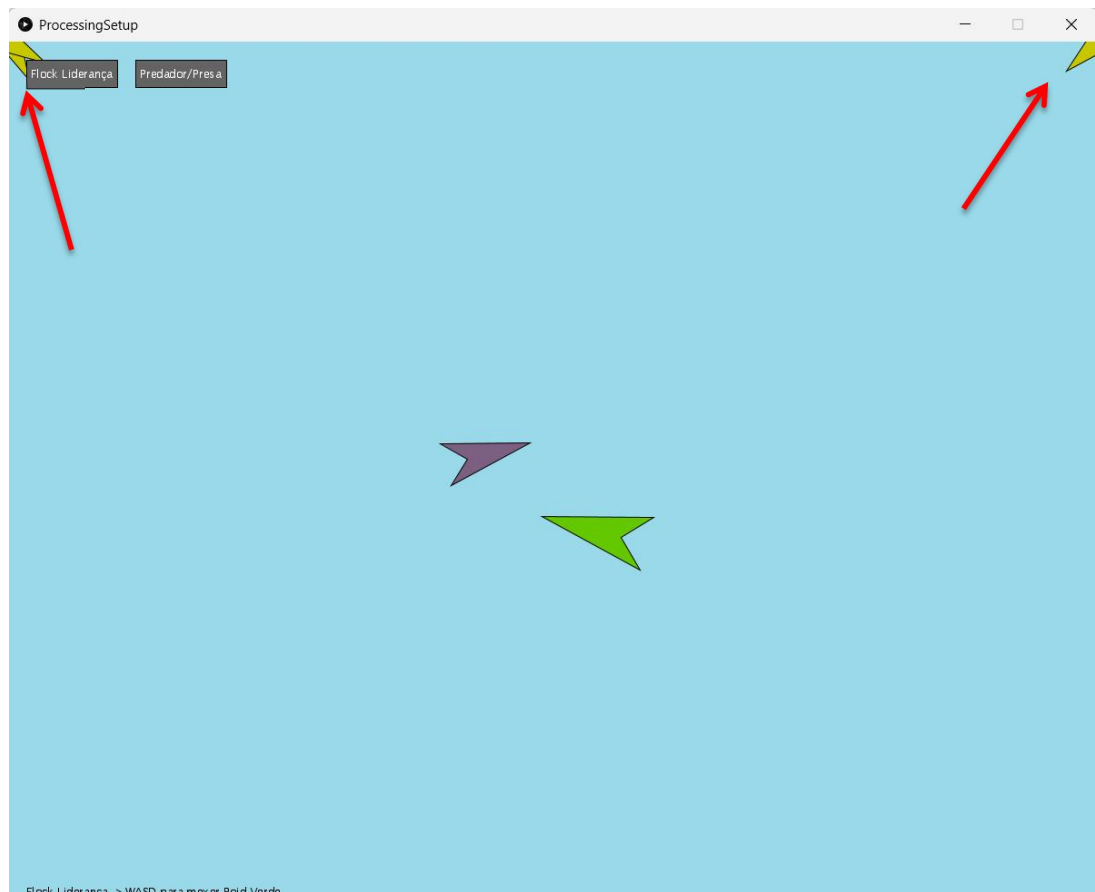


Figura 14 - Erro de boids a "piscar"

Exercício 3 - Flock com liderança

Este exercício segue a filosofia do anterior, mas temos outro botão para o comportamento.

O líder surge a verde e é controlado pelo *user* pelas teclas WASD. Para a inicialização foi feito o seguinte:

```
@Override
public void setup(PApplet p) {
    plt = new SubPlot(window, viewport, p.width, p.height);
    flock = new Flock(5, .1f, 0.8f, p.color(200, 200, 0) , sacWeights, p, plt);
    flock.getBoid(controlledBoidIndex).setShape(p, plt, p.color(100, 200, 0)); //lider
    allTrackingBodies = new ArrayList<Body>();
    predador = null;
    b = null;
}
```

No excerto acima colocamos uma cor verde ao líder para o destacar.

O controlo pelas teclas WASD foi feito através do `applyForce()`, com a entrada de um vetor com o valor de velocidade desejado:

```
@Override
public void keyPressed(PApplet p) {
    float speed = 3f;
    Boid controlledBoid = flock.getBoid(controlledBoidIndex); //lider

    if (p.key == 'w' || p.key == 'W') {
        controlledBoid.applyForce(new PVector(0, speed));
    }
    if (p.key == 's' || p.key == 'S') {
        controlledBoid.applyForce(new PVector(0, -speed));
    }
    if (p.key == 'a' || p.key == 'A') {
        controlledBoid.applyForce(new PVector(-speed, 0));
    }
    if (p.key == 'd' || p.key == 'D') {
        controlledBoid.applyForce(new PVector(speed, 0));
    }
    flock.setFollowTarget(controlledBoid.getPos());
}
```

E o tratamento de dados ao clicar no botão, consiste em remover os comportamentos atuais do flock e atribuir novos pesos para cada comportamento, para que a realização da dinâmica flock de Liderança seja executada:

```
public void mousePressed(PApplet p) {
    Boid aux = flock.getBoidMouse(p);
    int i = flock.getBoidIndex(aux);
    //Flock liderança
    if (BoidApp.isMouseOverButton(p, margin, buttonWidth, margin, margin+buttonHeight)) {
        float[] weights = {1f, 1f, 1f, 0f, 0f};
        flock.removeAllBehavior();
        flock.setBehaviorAllBoids(weights);
        predador = null;
        resetColors(p, plt);
    }

    (...)
}
```

Neste exercício não houve erro detetados e tudo correu conforme previsto.



Figura 15 - Print da FlockTextApp com um líder

Exercício 4 - Debugging

Não houve muitas alterações em relação ao que já fora feito anteriormente. Apenas se dividiu o ecrã em três viewports, cada um para o teste diferente (um *flock* com um *boid* a fazer *debugging*, onde o seu campo de visão é visível, tal como o seu vetor da velocidade e altera-se os *boids* visíveis por ele para a cor vermelha, um teste de Wander e Pursuiter, um teste de Seek e Flee.

Se clicarmos na tecla "T" o comportamento do boid presente no viewport do Seek/Flee muda o seu comportamento e ao clicarmos nalgum sítio do viewport o target é demonstrado por um pequeno círculo preto.

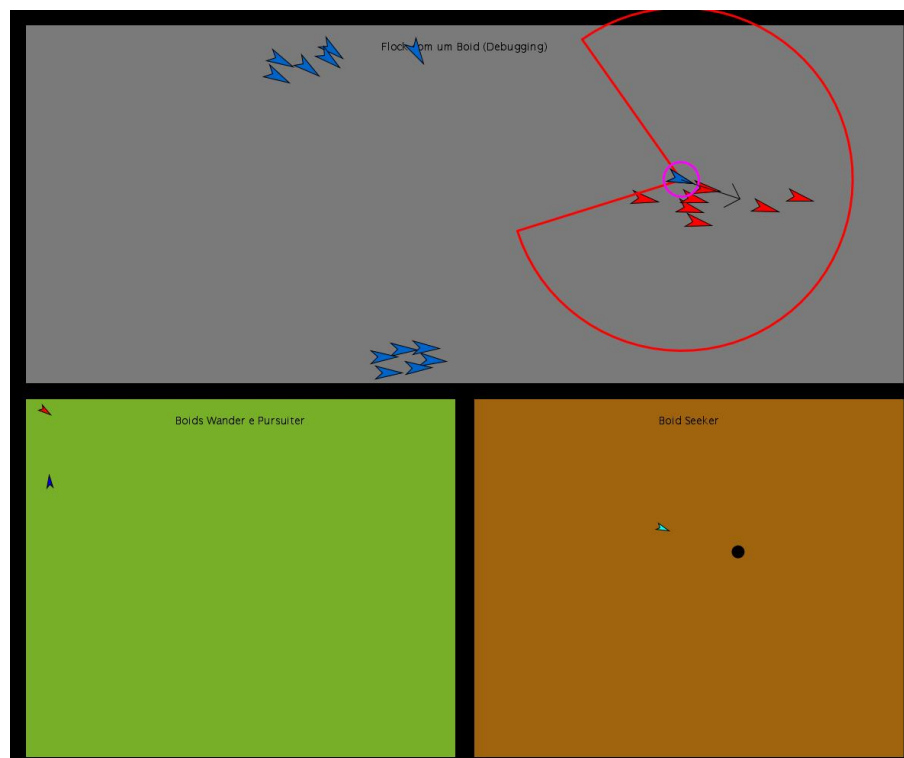


Figura 16 - Print da FlockDebuggingApp

Conclusão

Ao longo deste trabalho prático, analisamos o comportamento de um corpo em queda livre até a implementação de agentes autónomos com comportamentos individuais e de grupo.

Na primeira parte, utilizamos a ferramenta *Silico* para analisar a queda livre com atrito, através de diagramas. Em seguida, no sistema solar e no sistema de partículas, desenvolvemos classes e aplicações para a simulação do sistema, que foi bem sucedida.

No desenvolvimento do Boid com acelerador e travão, o comportamento só não foi perfeito, devido ao facto de que o target às vezes não alcança o alvo, embora vá na sua direção, devido às forças aplicadas.

A perseguição de uma presa por um predador também tiveram uma implementação praticamente perfeita, só tendo como problema os boids a “piscar”.

Finalmente, através de uma abordagem de *debugging*, verificámos a precisão e eficácia das simulações.

Bibliografia

Consulta:

Vieira, Paulo. (2023 - 2024). *Física e Movimento* - Moodle

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle

Diagramas Silico:

B1 - <https://silico.app/@mikos2004/onboardingexample/app?path=54>

B2 - <https://silico.app/@mikos2004/onboardingexample/app?path=92>

B3 - <https://silico.app/@mikos2004/onboardingexample/app?path=75>

B4 - <https://silico.app/@mikos2004/onboardingexample/app?path=107>